

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

-- Question 1
-- A
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name asc;

--B
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities

```

Query Result

SQL | Fetched 50 rows in 0.028 seconds

ACTOR_ID	FIRST_NAME	LAST_NAME
1	58 CHRISTIAN	AKROYD
2	182 DEBBIE	AKROYD
3	92 KIRSTEN	AKROYD
4	118 CUBA	ALLEN
5	145 KIM	ALLEN
6	194 MERYL	ALLEN
7	76 ANGELINA	ASTAIRE
8	112 RUSSELL	BACALL
9	190 AUDREY	BATLEY
10	67 JESSICA	BATLEY
11	115 HARRISON	BALE
12	187 RENEE	BALL
13	47 JULIA	BARRYMORE
14	158 VIVIAN	BASINGER
15	134 MUEL	BENTING

Line 6 Column 1 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

-- Question 1
-- A
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name asc;

--B
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities

```

Query Result

SQL | Fetched 50 rows in 0.03 seconds

ACTOR_ID	FIRST_NAME	LAST_NAME
1	92 KIRSTEN	AKROYD
2	182 DEBBIE	AKROYD
3	58 CHRISTIAN	AKROYD
4	194 MERYL	ALLEN
5	145 KIM	ALLEN
6	118 CUBA	ALLEN
7	76 ANGELINA	ASTAIRE
8	112 RUSSELL	BACALL
9	67 JESSICA	BATLEY
10	190 AUDREY	BATLEY
11	115 HARRISON	BALE
12	187 RENEE	BALL
13	47 JULIA	BARRYMORE
14	158 VIVIAN	BASINGER
15	134 MUEL	BENTING

Line 10 Column 41 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

-- Question 1
-- A
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name asc;

--B
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities

```

Query Result

SQL | Fetched 50 rows in 0.029 seconds

ACTOR_ID	FIRST_NAME	LAST_NAME
1	71 ADAM	GRANT
2	132 ADAM	HOPPER
3	165 AL	GARLAND
4	173 ALAN	DREYFUSS
5	146 ALBERT	JOHANSSON
6	125 ALBERT	NOLTE
7	29 ALEC	WAYNE
8	65 ANGELA	HUDSON
9	144 ANGELA	WITHERSPOON
10	76 ANGELINA	ASTAIRE
11	49 ANNE	CROWNN
12	190 AUDREY	BAILEY
13	34 AUDREY	OLIVIER
14	196 BELA	WALKEN
15	153 BEN	WARTON

Line 15 Column 40 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

-- Question 1
-- A
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name asc;

--B
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities

```

Query Result

SQL | All Rows Fetched: 1 in 0.017 seconds

2*3	
1	6

Line 20 Column 1 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

-- Question 1
-- A
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name asc;

--B
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities

```

Query Result x Query Result 1 x Query Result 2 x

SQL All Rows Fetched: 1 in 0.02 seconds

5+7
12

Line 22 Column 15 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
 - COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
 - CUSTOMER
 - FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

  From ACTOR
 Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
  From ACTOR
 Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
  From Dual;

Select 5 + 7
  From Dual;

-- Question 3
Select count(city_id) AS total_cities
  From City;

Select country_id, count(city_id) AS total_cities
  From City
 Group by country_id;

-- Question 4
Select country_id, count(city_id) AS total_cities
  From City
 Group by country_id
 Having count(city_id) > 50;

```

Query Result x Query Result 1 x Query Result 2 x

SQL All Rows Fetched: 1 in 0.032 seconds

TOTAL_CITIES
1 600

Line 27 Column 1 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

From ACTOR
Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
From ACTOR
Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
From Dual;

Select 5 + 7
From Dual;

-- Question 3
Select count(city_id) AS total_cities
From City;

Select country_id, count(city_id) AS total_cities
From City
Group by country_id;

-- Question 4
Select country_id, count(city_id) AS total_cities
From City
Group by country_id
Having count(city_id) > 50;
  
```

Query Result 1 x Query Result 2 x

SQL | Fetched 50 rows in 0.014 seconds

COUNTRY_ID	TOTAL_CITIES
1	1
2	3
3	1
4	2
5	1
6	13
7	1
8	1
9	3
10	2
11	1
12	3
13	2
14	2
15	50

Line 31 Column 1 | Insert | Modified | Unix/Mac: LF

Connections

- CATEGORY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
 - FILM_ACTOR
 - FILM_CATEGORY
 - FILM_TEXT
 - INVENTORY
 - LANGUAGE
 - PAYMENT
 - RENTAL

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page Lab6 CITY

Worksheet Query Builder

```

From ACTOR
Order by last_name asc, first_name desc;

-- C
Select Actor_ID, First_name, Last_name
From ACTOR
Order by First_name asc, Last_name asc;

-- Question 2
Select 2 * 3
From Dual;

Select 5 + 7
From Dual;

-- Question 3
Select count(city_id) AS total_cities
From City;

Select country_id, count(city_id) AS total_cities
From City
Group by country_id;

-- Question 4
Select country_id, count(city_id) AS total_cities
From City
Group by country_id
Having count(city_id) > 50;
  
```

Query Result 1 x Query Result 2 x

SQL | All Rows Fetched: 2 in 0.017 seconds

COUNTRY_ID	TOTAL_CITIES
1	23
2	60

Line 36 Column 28 | Insert | Modified | Unix/Mac: LF

Connections: Oracle Connections, Lab5b, Lab6

SQL Worksheet: History, Lab6

Worksheet: Query Builder

```

Order by last_name asc, first_name desc;

-- Question 1
Select Actor_ID, First_name, Last_name
From ACTOR
Order by First_name asc, Last_name asc;

-- Question 2
Select 2 + 3
From Dual;

Select 5 + 7
From Dual;

-- Question 3
Select count(city_id) AS total_cities
From City;

Select country_id, count(city_id) AS total_cities
From City
Group by country_id;

-- Question 4
Select country_id, count(city_id) AS total_cities
From City
Group by country_id
Having count(city_id) > 50;

Select country_id, count(city_id) AS total_cities
From City
Group by country_id
Having count(city_id) > 50

```

Query Result: All Rows Fetched: 2 in 0.021 seconds

COUNTRY_ID	TOTAL_CITIES
1	23
2	44

Connections: CITY, COUNTRY, CUSTOMER, FILM, FILM_ID, TITLE, DESCRIPTION, RELEASE_YEAR, LANGUAGE_ID, ORIGINAL_LANGUAGE_ID, RENTAL_DURATION, RENTAL_RATE, LENGTH, REPLACEMENT_COST, RATING, SPECIAL_FEATURES, LAST_UPDATE, FILM_ACTOR, FILM_CATEGORY, FILM_TEXT, INVENTORY, LANGUAGE

SQL Worksheet: History, Lab6

Worksheet: Query Builder

```

Order By count(city_id) asc;

-- Question 5
Select count(film_id), release_year
From film
Group by release_year;

Select count(film_id), release_year, rating
From film
Where rating in ('PG')
Group by release_year, rating;

Select count(film_id), release_year
From film
Where release_year > 2015
Group by release_year;

Select count(film_id), release_year
From film
Group by release_year
Having count(film_id) > 50;

Select count(film_id), release_year, rating
From film
Where release_year = 2015
Group by release_year, rating;

Select round(avg(rental_rate), 2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
From film
Group by rating;

```

Query Result: All Rows Fetched: 26 in 0.029 seconds

COUNT(FILM_ID)	RELEASE_YEAR
1	37 2022
2	46 2004
3	55 2002
4	42 2013
5	38 2010
6	34 2016
7	40 2020
8	38 2005
9	30 2009
10	44 2014
11	24 2000
12	34 2007
13	39 2021
14	34 2024
15	22 2025
16	43 2011
17	40 2006

Connections

- CITY
- COUNTRY
- CUSTOMER
- FILM
 - FILM_ID
 - TITLE
 - DESCRIPTION
 - RELEASE_YEAR
 - LANGUAGE_ID
 - ORIGINAL_LANGUAGE_ID
 - RENTAL_DURATION
 - RENTAL_RATE
 - LENGTH
 - REPLACEMENT_COST
 - RATING
 - SPECIAL_FEATURES
 - LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Order By count(city_id) asc;

-- Question 5
Select count(film_id), release_year
  From film
 Group by release_year;

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year,rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;
  
```

Query Result | Script Output | Query Result 1

SQL | All Rows Fetched: 26 in 0.049 seconds

COUNT(FILM_ID)	RELEASE_YEAR	RATING
1	8 2022	PG
2	12 2002	PG
3	12 2010	PG
4	6 2013	PG
5	6 2004	PG
6	8 2005	PG
7	10 2016	PG
8	10 2020	PG
9	7 2009	PG
10	1 2000	PG
11	5 2014	PG
12	7 2007	PG
13	7 2021	PG
14	10 2024	PG
15	4 2025	PG
16	9 2006	PG
17	6 2011	PG

Line 53 Column 30 | Insert | Modified | Unix/Mac: LF

Connections

- CITY
- COUNTRY
- CUSTOMER
- FILM
 - FILM_ID
 - TITLE
 - DESCRIPTION
 - RELEASE_YEAR
 - LANGUAGE_ID
 - ORIGINAL_LANGUAGE_ID
 - RENTAL_DURATION
 - RENTAL_RATE
 - LENGTH
 - REPLACEMENT_COST
 - RATING
 - SPECIAL_FEATURES
 - LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Order By count(city_id) asc;

-- Question 5
Select count(film_id), release_year
  From film
 Group by release_year;

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year,rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;
  
```

Query Result | Script Output | Query Result 1

SQL | All Rows Fetched: 10 in 0.05 seconds

COUNT(FILM_ID)	RELEASE_YEAR
1	40 2019
2	47 2018
3	40 2017
4	33 2023
5	37 2022
6	34 2016
7	40 2020
8	39 2021
9	34 2024
10	22 2025

Line 58 Column 23 | Insert | Modified | Unix/Mac: LF

Connections

- CITY
- COUNTRY
- CUSTOMER
- FILM
 - FILM_ID
 - TITLE
 - DESCRIPTION
 - RELEASE_YEAR
 - LANGUAGE_ID
 - ORIGINAL_LANGUAGE_ID
 - RENTAL_DURATION
 - RENTAL_RATE
 - LENGTH
 - REPLACEMENT_COST
 - RATING
 - SPECIAL_FEATURES
 - LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Order By count(city_id) asc;

-- Question 5
Select count(film_id), release_year
  From film
 Group by release_year;

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year > 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;

```

Lab6 : Select count(film_id), release_year From film Group by release_year Having count(film_id) > 50

Query Result | SQL | All Rows Fetched: 1 in 0.043 seconds

COUNT(FILM_ID)	RELEASE_YEAR
1	55 2002

Line 63 Column 28 | Insert | Modified | Unix/Mac: LF

Connections

- CITY
- COUNTRY
- CUSTOMER
- FILM
 - FILM_ID
 - TITLE
 - DESCRIPTION
 - RELEASE_YEAR
 - LANGUAGE_ID
 - ORIGINAL_LANGUAGE_ID
 - RENTAL_DURATION
 - RENTAL_RATE
 - LENGTH
 - REPLACEMENT_COST
 - RATING
 - SPECIAL_FEATURES
 - LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Order By count(city_id) asc;

-- Question 5
Select count(film_id), release_year
  From film
 Group by release_year;

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year > 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;

```

Query Result | SQL | All Rows Fetched: 5 in 0.029 seconds

COUNT(FILM_ID)	RELEASE_YEAR	RATING
1	8 2015	G
2	6 2015	PG
3	16 2015	NC-17
4	12 2015	R
5	7 2015	PG-13

Line 69 Column 31 | Insert | Modified | Unix/Mac: LF

Connections

- CITY
- COUNTRY
- CUSTOMER
- FILM
 - FILM_ID
 - TITLE
 - DESCRIPTION
 - RELEASE_YEAR
 - LANGUAGE_ID
 - ORIGINAL_LANGUAGE_ID
 - RENTAL_DURATION
 - RENTAL_RATE
 - LENGTH
 - REPLACEMENT_COST
 - RATING
 - SPECIAL_FEATURES
 - LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet History

Worksheet | Query Builder

```

Order by count(city_id) asc;

-- Question 5
Select count(film_id), release_year
  From film
 Group by release_year;

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;
  
```

Query Result | Script Output | Query Result 1

SQL | All Rows Fetched: 5 in 0.033 seconds

	AVG_RENTAL_RATE	MAX_RENTAL_DURATION	RATING
1	2.89	7G	
2	3.83	7PG-13	
3	2.94	7R	
4	2.97	7NC-17	
5	3.05	7PG	

| Line 73 Column 17 | Insert | Modified | Unix/Mac: LF

Connections

- ACTOR
- ADDRESS
- CATEGORY
- CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT
- RENTAL
- STAFF

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet History

Worksheet | Query Builder

```

Select count(film_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(film_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(film_id), release_year
  From film
 Group by release_year
 Having count(film_id) > 50;

Select count(film_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;

-- Question 6
Select city.city, country.country
  From city, country
 Where city.country_id = country.country_id;

Select city.city, country.country
  From city JOIN country on city.country_id = country.country_id;

Select city.city, country.country -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From city natural join country;
  
```

Query Result | Script Output | Query Result 1

SQL | Fetched 50 rows in 0.05 seconds

CITY	COUNTRY
1 Jos Azueta	Mexico
2 Juazeiro do Norte	Brazil
3 Juiz de Fora	Brazil
4 Junan	China
5 Jurez	Mexico
6 Kabul	Afghanistan
7 Kaduna	Nigeria
8 Kakamigahara	Japan
9 Kaliningrad	Russian Federation
10 Kalisz	Poland
11 Kamakura	Japan
12 Kamarhati	India
13 Kanjanets-Podilskyi	Ukraine
14 Kamyin	Russian Federation
15 Kanazawa	Japan

(4 more...) | Line 77 Column 44 | Insert | Modified | Unix/Mac: LF

Connections

- ACTOR
- ADDRESS
- CATEGORY
- CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT
- RENTAL
- STAFF

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Select count(file_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(file_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(file_id), release_year
  From film
 Group by release_year
 Having count(file_id) > 50;

Select count(file_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;

-- Question 6
Select city.city, country.country
  From city, country
 Where city.country_id = country.country_id;

Select city.city, country.country
  From city INNER JOIN country on city.country_id = country.country_id;

Select city.city, country.country -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From city natural join country;

```

Query Result | Script Output | Query Result 1

SQL | Fetched 50 rows in 0.025 seconds

CITY	COUNTRY
1 A Corua (La Corua)	Spain
2 Abha	Saudi Arabia
3 Abu Dhabi	United Arab Emirates
4 Acua	Mexico
5 Adana	Turkey
6 Addis Abeba	Ethiopia
7 Aden	Yemen
8 Adoni	India
9 Ahmadnagar	India
10 Akishima	Japan
11 Akron	United States
12 al-Ayn	United Arab Emirates
13 al-Hawiyah	Saudi Arabia
14 al-Manama	Bahrain
15 al-Qadisiyah	Sudan

(5 more...) | Line 80 Column 74 | Insert | Modified | Unix/Mac: LF

Connections

- ACTOR
- ADDRESS
- CATEGORY
- CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT
- RENTAL
- STAFF

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Scripts.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Select count(file_id), release_year, rating
  From film
 Where rating in ('PG')
 Group by release_year, rating;

Select count(file_id), release_year
  From film
 Where release_year > 2015
 Group by release_year;

Select count(file_id), release_year
  From film
 Group by release_year
 Having count(file_id) > 50;

Select count(file_id), release_year, rating
  From film
 Where release_year = 2015
 Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
  From film
 Group by rating;

-- Question 6
Select city.city, country.country
  From city, country
 Where city.country_id = country.country_id;

Select city.city, country.country
  From city INNER JOIN country on city.country_id = country.country_id;

Select city.city, country.country -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From city natural join country;

```

Query Result | Script Output | Query Result 1

SQL | All Rows Fetched: 0 in 0.099 seconds

CITY	COUN...
------	---------

(5 more...) | Line 83 Column 36 | Insert | Modified | Unix/Mac: LF

Connections

- ACTOR
- ADDRESS
- CATEGORY
- CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT
- RENTAL
- STAFF

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Where release_year > 2015
Group by release_year;

Select count(film_id), release_year
From film
Group by release_year
Having count(film_id) > 50;

Select count(film_id), release_year, rating
From film
Where release_year = 2015
Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
From film
Group by rating;

-- Question 6
Select city.city, country.country
From city, country
Where city.country_id = country.country_id;

Select city.city, country.country
From city INNER JOIN country on city.country_id = country.country_id;

Select city.city, country.country -- This returns a blank result this is because it tries to match column by column and that is not done properly
From city natural join country;

-- Question 7
Select country.country, city.city
From city INNER JOIN country on country.country_id = city.country_id;

Select country.country, count(city.city)
From city INNER JOIN country on country.country_id = city.country_id
Group by country.country;
  
```

Query Result | Script Output | Query Result 1

SQL | Fetched 50 rows in 0.02 seconds

COUNTRY	CITY
1 Spain	A Corua (La Corua)
2 Saudi Arabia	Abha
3 United Arab Emirates	Abu Dhabi
4 Mexico	Acua
5 Turkey	Adana
6 Ethiopia	Addis Abeba
7 Yemen	Aden
8 India	Adoni
9 India	Ahmadnagar
10 Japan	Akishima
11 United States	Akron
12 United Arab Emirates	al-Ayn
13 Saudi Arabia	al-Hawiya
14 Bahrain	al-Manama
15 Sudan	al-Radarif

(4 more...) | Line 87 Column 74 | Insert | Modified | Unix/Mac: LF

Connections

- ACTOR
- ADDRESS
- CATEGORY
- CITY
 - CITY_ID
 - CITY
 - COUNTRY_ID
 - LAST_UPDATE
- COUNTRY
 - COUNTRY_ID
 - COUNTRY
 - LAST_UPDATE
- CUSTOMER
- FILM
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT
- RENTAL
- STAFF

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Where release_year > 2015
Group by release_year;

Select count(film_id), release_year
From film
Group by release_year
Having count(film_id) > 50;

Select count(film_id), release_year, rating
From film
Where release_year = 2015
Group by release_year, rating;

Select round(avg(rental_rate),2) AS avg_rental_rate, max(rental_duration) AS max_rental_duration, rating
From film
Group by rating;

-- Question 6
Select city.city, country.country
From city, country
Where city.country_id = country.country_id;

Select city.city, country.country
From city INNER JOIN country on city.country_id = country.country_id;

Select city.city, country.country -- This returns a blank result this is because it tries to match column by column and that is not done properly
From city natural join country;

-- Question 7
Select country.country, city.city
From city INNER JOIN country on country.country_id = city.country_id;

Select country.country, count(city.city)
From city INNER JOIN country on country.country_id = city.country_id
Group by country.country;
  
```

Query Result | Script Output | Query Result 1

SQL | Fetched 50 rows in 0.028 seconds

COUNTRY	COUNT(CITY.CITY)
1 Mozambique	3
2 Myanmar	2
3 Poland	8
4 Saudi Arabia	5
5 Israel	4
6 Malaysia	3
7 Morocco	3
8 Nepal	1
9 Runion	1
10 Argentina	13
11 Dominican Republic	3
12 Moldova	1
13 Russian Federation	20
14 Spain	5
15 Taiwan	10

(5 more...) | Line 91 Column 26 | Insert | Modified | Unix/Mac: LF

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

Question 7

```

Select country.country, city.city
  From city INNER JOIN Country on country.country_id = city.country_id;

Select country.country, count(city.city)
  From city INNER JOIN Country on country.country_id = city.country_id
 Group by country.country;

```

Question 8

```

Select film.title, language.name
  From film, language
 Where film.original_language_id = language.language_id;

Select film.title, language.name
  From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id), 'No Original Language Reported') AS Original_Language
  From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.021 seconds

	TITLE	NAME
1	MUSSOLINI SPOILERS	English
2	MYSTIC TRUMAN	English
3	NAME DETECTIVE	Japanese
4	NASH CHOCOLAT	Italian
5	NATIONAL STORY	Japanese
6	NATURAL STOCK	Japanese
7	NECKLACE OUTBREAK	Japanese
8	NEIGHBORS CHARADE	English
9	NEMO CAMPUS	Mandarin
10	NETWORK PEAK	Mandarin
11	NEWSIES STORY	Italian
12	NEWTON LABYRINTH	Italian
13	NIGHTMARE CHILL	Mandarin
14	NONE SPIKING	French
15	NOON PAPI	French
16	NORTH TEQUILA	Italian
17	NORTHWEST POLISH	Mandarin

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

Question 7

```

Select country.country, city.city
  From city INNER JOIN Country on country.country_id = city.country_id;

Select country.country, count(city.city)
  From city INNER JOIN Country on country.country_id = city.country_id
 Group by country.country;

```

Question 8

```

Select film.title, language.name
  From film, language
 Where film.original_language_id = language.language_id;

Select film.title, language.name
  From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id), 'No Original Language Reported') AS Original_Language
  From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.022 seconds

	TITLE	NAME
1	GROOVE FICTION	Mandarin
2	GROSSE WONDERFUL	Japanese
3	GROUNDHOG UNCUT	French
4	GUMP DATE	Mandarin
5	GUN BONNIE	Mandarin
6	GUNFIGHT MOON	French
7	GUNFIGHTER MUSSOLINI	French
8	GUYS FALCON	French
9	HALF OUTFIELD	Mandarin
10	HALLOWEEN NUTS	Italian
11	HAMLET WISDOM	Italian
12	HANDICAP BOONDOCK	English
13	HANGING DEEP	Mandarin
14	HANKY OCTOBER	Italian
15	HANOVER GALAXY	Japanese
16	HAPPINESS UNITED	Mandarin
17	HARDLY ROBBERS	Japanese

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

-- Question 7
Select country.country, city.city
  From city INNER JOIN Country on country.country_id = city.country_id;

Select country.country, count(city.city)
  From city INNER JOIN Country on country.country_id = city.country_id
 Group by country.country;

-- Question 8
Select film.title, language.name
  From film, language
 Where film.original_language_id = language.language_id;

Select film.title, language.name
  From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id), 'No Original Language Reported') AS Original_Language
  From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name

```

Script Output | Query Result | Query Result 1

SQL | All Rows Fetched: 0 in 0.106 seconds

TITLE	NAME
-------	------

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

  From city INNER JOIN Country on country.country_id = city.country_id
 Group by country.country;

-- Question 8
Select film.title, language.name
  From film, language
 Where film.original_language_id = language.language_id;

Select film.title, language.name
  From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
  From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id), 'No Original Language Reported') AS Original_Language
  From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name
  From film LEFT OUTER JOIN language on film.original_language_id = language.language_id
 Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
  From film RIGHT OUTER JOIN language on film.original_language_id = language.language_id;

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.035 seconds

TITLE	NAME	ORIGINAL_LANGUAGE
1 GROOVE FICTION	Mandarin	4
2 GROSSE WONDERFUL	Japanese	3
3 GROUNDHOG UNCAT	French	5
4 GUMP DATE	Mandarin	4
5 GUN BONNIE	Mandarin	4
6 GUNFIGHT MOON	French	5
7 GUNFIGHTER MUSSOLINI	French	5
8 GUYS FALCON	French	5
9 HALF OUTFIELD	Mandarin	4
10 HALL CASSIDY	(null)	No Original Language Reported
11 HALLOWEEN NUTS	Italian	2
12 HAMLET WISDOM	Italian	2
13 HANDICAP BOONDOCK	English	1
14 HANGING DEEP	Mandarin	4
15 HANKY OCTOBER	Italian	2
16 HANOVER GALAXY	Japanese	3
17 HAPPINESS UNITED	Mandarin	4

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

From city INNER JOIN country on country.country_id = city.country_id
Group by country.country;

-- Question 8
Select film.title, language.name
From film, language
Where film.original_language_id = language.language_id;

Select film.title, language.name
From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id, 'No Original Language Reported') AS Original_Language
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id
Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
From film RIGHT OUTER JOIN language on film.original_language_id = language.language_id;

```

Script Output | Query Result | Query Result 1

SQL | All Rows Fetched: 13 in 0.032 seconds

TITLE	NAME
1 TRADING PINOCCHIO	(null)
2 DOWNHILL ENOUGH	(null)
3 LADY STAGE	(null)
4 BUCKET BROTHERHOOD	(null)
5 HALL CASSIDY	(null)
6 ZOOLANDER FICTION	(null)
7 ANYTHING SAVANNAH	(null)
8 BERETS AGENT	(null)
9 HOMICIDE PEACH	(null)
10 PERFECT GROOVE	(null)
11 SLIPPER FIDELITY	(null)
12 WANDA CHAMBER	(null)
13 SECRETARY ROUGE	(null)

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Select film.title, language.name
From film, language
Where film.original_language_id = language.language_id;

Select film.title, language.name
From film INNER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id, 'No Original Language Reported') AS Original_Language
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id
Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
From film RIGHT OUTER JOIN language on film.original_language_id = language.language_id;

Select language.name
from language LEFT OUTER JOIN film on film.language_id = language.language_id
Where film.language_id is null
Group by language.name;

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.036 seconds

TITLE	NAME
1 FINDING ANACONDA	Japanese
2 FLAMINGOS CONNECTICUT	Japanese
3 FLASH WARS	Japanese
4 FLATLINERS KILLER	Japanese
5 FLINTSTONES HAPPINESS	Japanese
6 FRANKENSTEIN STRANGER	Japanese
7 FREAKY POCUS	Japanese
8 FREDDY STORM	Japanese
9 EMPIRE MALKOVICH	Japanese
10 ESCAPE METROPOLIS	Japanese
11 NAME DETECTIVE	Japanese
12 NATIONAL STORY	Japanese
13 NATURAL STOCK	Japanese
14 NECKLACE OUTBREAK	Japanese
15 NOTTING SPEAKEASY	Japanese
16 OCTOBER SUBMARINE	Japanese
17 CHICKEN HELL FIGHTERS	Japanese

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Select film.title, language.name -- This returns a blank result this is because it tries to match column by column and that is not done properly
From film NATURAL JOIN language;

Select film.title, language.name, NVL(TO_CHAR(film.original_language_id), 'No Original Language Reported') AS Original_Language
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id;

Select film.title, language.name
From film LEFT OUTER JOIN language on film.original_language_id = language.language_id
Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
From film RIGHT OUTER JOIN language on film.original_language_id = language.language_id;

Select language.name
From language LEFT OUTER JOIN film on film.language_id = language.language_id
Where film.language_id is null
Group by language.name;

-- Question 9
Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

Select film.title, count(inventory_id), inventory.store_id

```

Script Output | Query Result 1 x

SQL | All Rows Fetched: 7 in 0.03 seconds

NAME
1 Italian
2 Spanish
3 French
4 Arabic
5 Japanese
6 German
7 Mandarin

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

From film LEFT OUTER JOIN language on film.original_language_id = language.language_id
Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
From film RIGHT OUTER JOIN language on film.original_language_id = language.language_id;

Select language.name
From language LEFT OUTER JOIN film on film.language_id = language.language_id
Where film.language_id is null
Group by language.name;

-- Question 9
Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

Select film.title, count(inventory_id), inventory.store_id
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

```

Script Output | Query Result 1 x

SQL | Fetched 50 rows in 0.038 seconds

TITLE	COUNT(INVENTORY_ID)
1 DESTINY SATURDAY	5
2 SHREK LICENSE	3
3 PATTON INTERVIEW	7
4 CLYDE THEORY	3
5 GENTLEMEN STAGE	6
6 ATTACKS HATE	5
7 WORDS HUNTER	4
8 GHOST GROUNDHOG	5
9 SLEEPY JAPANESE	5
10 WITCHES PANIC	7
11 SAGEBRUSH CLUELESS	5
12 SHAKESPEARE SADDLE	6
13 DONNIE ALLEY	4
14 HANGING DEEP	4
15 POTLUCK MIXED	3
16 CONFIDENTIAL INTERVIEW	8
17 PRIDE ALAMO	5

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

From film LEFT OUTER JOIN language ON film.original_language_id = language.language_id
Where film.original_language_id is null;

Select NVL(film.title, 'N/A') as Title, language.name
From film RIGHT OUTER JOIN language ON film.original_language_id = language.language_id;

Select language.name
from language LEFT OUTER JOIN film ON film.language_id = language.language_id
Where film.language_id is null
Group by language.name;

-- Question 9
Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

Select film.title, count(inventory_id), inventory.store_id
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.035 seconds

TITLE	COUNT(INVENTORY_ID)	STORE_ID
1 ACADEMY DINOSAUR	4	1
2 AFFAIR PREJUDICE	4	1
3 AGENT TRUMAN	3	1
4 AIRPLANE SIERRA	2	1
5 ALABAMA DEVIL	3	1
6 ALADDIN CALENDAR	4	1
7 ALAMO VIDEOTAPE	4	1
8 ALASKA PHANTOM	3	1
9 ALIEN CENTER	2	1
10 ALLEY EVOLUTION	2	1
11 ALONE TRIP	3	1
12 ALTER VICTORY	3	1
13 AMADEUS HOLY	4	1
14 AMELIE HELFIGHTERS	3	1
15 AMERICAN CIRCUS	2	1
16 AMISTAD MIDSUMMER	4	1
17 ANACONDA CONFESSIONS	3	1

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Group by language.name;

-- Question 9
Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

Select film.title, count(inventory_id), inventory.store_id
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer join inventory on film.film_id = inventory.film_id
Group by film.title;

-- Question 10
SELECT film.title, category.name
FROM film_category INNER JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;

SELECT film.title, category.name
FROM film_category LEFT JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.038 seconds

TITLE	COUNT(INVENTORY_ID)
1 TROUBLE DATE	6
2 UNFORGIVEN ZOOLANDER	5
3 TROOPERS METAL	6
4 ENCOUNTERS CURTAIN	6
5 MODERN DORADO	4
6 LANGUAGE COMBOY	3
7 UNFAITHFUL KILL	3
8 GORGEOUS BINGO	5
9 KING EVOLUTION	2
10 KNOCK WARLOCK	7
11 CHEAPER CLYDE	5
12 SWEETHEARTS SUSPECTS	8
13 EVE RESURRECTION	2
14 OUTBREAK DIVINE	5
15 MIXED DOORS	2
16 OUTILLS BULL	4
17 CADDYSHACK 1EDT	4

Waiting for Query Result to finish (2 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet History

Worksheet | Query Builder

```

Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer Join inventory on film.film_id = inventory.film_id
Group by film.title;

-- Question 10
SELECT film.title, category.name
FROM film_category INNER JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;

SELECT film.title, category.name
FROM film_category LEFT JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;
--where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.097 seconds

TITLE	NAME
1 VANILLA DAY	Games
2 YOUNG LANGUAGE	Documentary
3 INSECTS STONE	Animation
4 INTRIGUE WORST	Foreign
5 JACKET FRISCO	Drama
6 LAMBS CINCINATTI	Games
7 LESSON CLEOPATRA	Sports
8 LOLA AGENT	Horror
9 ORIENT CLOSER	Drama
10 OTHERS SOUP	Travel
11 PACKER MADIGAN	Animation
12 PANIC CLUB	Horror
13 PEARL DESTINY	Foreign
14 PHILADELPHIA WIFE	Sports
15 RAGE GAMES	Family
16 REAR TRADING	Action
17 REININTON MITCHES	Drama

Waiting for Query Result to finish (2 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet History

Worksheet | Query Builder

```

Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer Join inventory on film.film_id = inventory.film_id
Group by film.title;

-- Question 10
SELECT film.title, category.name
FROM film_category INNER JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;

SELECT film.title, category.name
FROM film_category LEFT JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;
--where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.04 seconds

TITLE	NAME
1 VANILLA DAY	Games
2 YOUNG LANGUAGE	Documentary
3 INSECTS STONE	Animation
4 INTRIGUE WORST	Foreign
5 JACKET FRISCO	Drama
6 LAMBS CINCINATTI	Games
7 LESSON CLEOPATRA	Sports
8 LOLA AGENT	Horror
9 ORIENT CLOSER	Drama
10 OTHERS SOUP	Travel
11 PACKER MADIGAN	Animation
12 PANIC CLUB	Horror
13 PEARL DESTINY	Foreign
14 PHILADELPHIA WIFE	Sports
15 RAGE GAMES	Family
16 REAR TRADING	Action
17 REININTON MITCHES	Drama

Waiting for Query Result to finish (2 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

Group by film.title, inventory.store_id
Order by inventory.store_id;

Select film.title, count(inventory_id)
From film left outer Join inventory on film.film_id = inventory.film_id
Group by film.title;

-- Question 10
SELECT film.title, category.name
FROM film_category INNER JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;

SELECT film.title, category.name
FROM film_category LEFT JOIN category ON film_category.category_id = category.category_id
INNER JOIN film ON film_category.film_id = film.film_id;
where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.026 seconds

TITLE	NAME
1 USUAL UNTOUCHABLES	Foreign
2 VACATION BOONDOCK	Sci-Fi
3 VALENTINE VANISHING	Travel
4 VALLEY PACKER	Comedy
5 VAMPIRE WHALE	New
6 VANILLA DAY	Games
7 VANISHED GARDEN	New
8 VANISHING ROCKY	Music
9 VARSITY TRIP	New
10 VELVET TERMINATOR	Comedy
11 VERTIGO NORTHWEST	Comedy
12 VICTORY ACADEMY	Sports
13 VIDEOTAPE ARSENIC	Games
14 VIETNAM SMOOCHY	Drama
15 VILLAIN DESPERATE	Documentary
16 VIRGIN DAISY	Drama
17 VIRGINIAN PLUTO	Documentary

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```

--where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film INNER JOIN inventory ON film.film_id = inventory.film_id
LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
GROUP BY film.title, category.name;

```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.036 seconds

TITLE	NAME	COUNT(INVENTORY.INVENTORY_ID)
1 SENSE GREEK	Sports	2
2 SILVERADO GOLDFINGER	Sci-Fi	4
3 SNATCH SLIPPER	Comedy	7
4 SOLDIERS EVOLUTION	Sci-Fi	2
5 SPEED SUIT	Travel	2
6 SPIRITED CASUALTIES	Sci-Fi	3
7 INSIDER ARIZONA	Music	5
8 INSTINCT AIRPORT	Sports	5
9 JUNGLE CLOSER	New	2
10 LESSON CLEOPATRA	Sports	2
11 LOST BIRD	Foreign	3
12 MOB DUFFEL	Drama	4
13 MODERN DORADO	Documentary	4
14 MONEY HAROLD	New	7
15 MOULIN WAKE	Travel	5
16 NATURAL STOCK	Family	3
17 NOVOCATINE FLIGHT	Snorts	5

Waiting for Query Result to finish (2 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```
--where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film INNER JOIN inventory ON film.film_id = inventory.film_id
LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
GROUP BY film.title, category.name;
```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.037 seconds

TITLE	NAME	COUNT(INVENTORY.INVENTORY_ID)
1 DARKO DORADO	Action	3
2 DESERT POSEIDON	Horror	6
3 EASY GLADIATOR	Action	5
4 EXPENDABLE STALLION	Documentary	8
5 EYES DRIVING	Sci-Fi	5
6 ALASKA PHANTOM	Music	7
7 CADDYSHACK JEDI	Action	4
8 GUNFIGHTER MUSSOLINI	Sports	4
9 SENSE GREEK	Sports	2
10 SILVERADO GOLDFINGER	Sci-Fi	4
11 SNATCH SLIPPER	Comedy	7
12 HAMLET WISDOM	Sci-Fi	4
13 HAUNTED ANTITRUST	Travel	2
14 LESSON CLEOPATRA	Sports	2
15 PRIMARY GLASS	Action	8
16 TEQUILA PAST	Children	2
17 TIMBERLAND SKY	Classics	7

Waiting for Query Result to finish (1 more...)

Connections

- FILM_ID
- TITLE
- DESCRIPTION
- RELEASE_YEAR
- LANGUAGE_ID
- ORIGINAL_LANGUAGE_ID
- RENTAL_DURATION
- RENTAL_RATE
- LENGTH
- REPLACEMENT_COST
- RATING
- SPECIAL_FEATURES
- LAST_UPDATE
- FILM_ACTOR
- FILM_CATEGORY
- FILM_TEXT
- INVENTORY
- LANGUAGE
- PAYMENT

Reports

- All Reports
- About Your Database
- All Objects
- Analytic View Reports
- Application Express
- ASH and AWR
- Database Administration
- Data Dictionary
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- PLSQL
- Security
- Streams
- Table
- TimesTen Reports
- User Defined Reports
- XML

Welcome Page | Script.sql

SQL Worksheet: History

Worksheet | Query Builder

```
--where film_category.category_id is null;

SELECT film.title, category.name
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id;

-- Question 11
SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film_category LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN film ON film_category.film_id = film.film_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
LEFT OUTER JOIN inventory ON film.film_id = inventory.film_id
GROUP BY film.title, category.name;

SELECT film.title, category.name, COUNT(inventory.inventory_id)
FROM film INNER JOIN inventory ON film.film_id = inventory.film_id
LEFT OUTER JOIN film_category ON film.film_id = film_category.film_id
LEFT OUTER JOIN category ON film_category.category_id = category.category_id
GROUP BY film.title, category.name;
```

Script Output | Query Result | Query Result 1

SQL | Fetched 50 rows in 0.036 seconds

TITLE	NAME	COUNT(INVENTORY.INVENTORY_ID)
1 ALASKA PHANTOM	Music	7
2 CADDYSHACK JEDI	Action	4
3 DARKO DORADO	Action	3
4 DESERT POSEIDON	Horror	6
5 DISCIPLE MOTHER	Travel	6
6 DIVORCE SHINING	Sports	2
7 DOCTOR GRAIL	Children	2
8 EASY GLADIATOR	Action	5
9 WORDS HUNTER	Music	4
10 GUNFIGHTER MUSSOLINI	Sports	4
11 HAMLET WISDOM	Sci-Fi	4
12 HAUNTED ANTITRUST	Travel	2
13 JUNGLE CLOSER	New	2
14 LESSON CLEOPATRA	Sports	2
15 LOST BIRD	Foreign	3
16 MOB DUFFEL	Drama	4
17 MODERN DORADO	Documentary	4

Waiting for Query Result to finish (2 more...)